

Laboratory Work 1: Relational Model & Keys

Course: Database Systems

Tools Required: ERDPlus, Microsoft Word / Google Docs

Submission Format: PDF Document

Part 1: Key Identification Exercises

Task 1.1: Superkey and Candidate Key Analysis

Relation A: Employee

Given Relation: Employee (EmpID, SSN, Email, Phone, Name, Department, Salary)

1. List at least 6 different superkeys:

1)EmpID 2)SSN 3)Email 4)EmpID, Name 5)SSN, Phone 6)EmpID, SSN, Email

2. Identify all candidate keys:

1) EmpID 2)SSN 3) Phone 4)Email

3. Which candidate key would you choose as primary key and why?

Chosen Primary Key: EmpID

Justification: It is a short, unique, numeric surrogate key specifically designed for internal database identification, whereas SSN is sensitive personal data and Email can change.

4. Can two employees have the same phone number? Justify your answer based on the data shown:

Yes, two employees can potentially share a phone number (e.g., shared corporate/department lines or shared residential phone numbers). Furthermore, phone numbers are optional or subject to updates, so Phone cannot guarantee absolute uniqueness or serve as a key attribute.

Relation B: Course Registration

- Determine the minimum attributes needed for the primary key:**

Primary Key: {StudentID, CourseCode, Semester, Year}

- Explain why each attribute in your primary key is necessary:**

StudentID: Identifies which specific student is registering.

- CourseCode: Identifies the course subject
- Semester & Year: Identifies the academic term, allowing students to retake courses or enroll across different semesters.
- **Identify any additional candidate keys:**
No additional candidate keys exist because no proper subset of these attributes uniquely identifies a registration instance under the stated business rules.

Task 1.2: Foreign Key Design

Given Tables: Student, Professor, Course, Department, Enrollment

Foreign Key Relationships:

1. Student(AdvisorID) → references Professor(ProfID)
2. Department(ChairID) → references Professor(ProfID)
3. Enrollment(StudentID) → references Student(StudentID)
4. Enrollment(CourseID) → references Course(CourseID)

Part 2: ER Diagram Construction

Task 2.1: Hospital Management System

1. **Entities & Classification:**
 - **Strong Entities:** Patient, Doctor, Department
 - **Weak Entity:** Hospital Room (dependent on Department)
2. **Attributes Classification:**
 - Patient: PatientID (Simple, Primary Key), Name (Simple), Birthdate (Simple), Address (Composite: Street, City, State, Zip), Phone (Multi-valued), InsuranceInfo (Simple).
 - Doctor: DoctorID (Simple, Primary Key), Name (Simple), Specialization (Multi-valued), Phone (Simple), OfficeLocation (Simple).
 - Department: DeptCode (Simple, Primary Key), Name (Simple), Location (Simple).
 - Hospital Room: RoomNumber (Partial Key), Capacity/Type (Simple).
3. **Relationships & Cardinalities:**
 - **Department contains Rooms:** 1:N (One Department has many Rooms; a Room belongs to exactly 1 Department).
 - **Appointments:** M:N relationship between Patient and Doctor (Attributes: Date, Time, Purpose, Notes).
 - **Prescriptions:** M:N relationship between Patient and Doctor (Attributes: Dosage, Instructions, Date).

Task 2.2: E-commerce Platform

1. **Weak Entity & Justification:**
 - **Weak Entity:** OrderItem
 - **Justification:** An OrderItem cannot exist independently without its parent Order. Its identity relies on the composite key {OrderID, ItemNumber}.

2. Many-to-Many Relationship with Attributes:

- Customer reviews Product (M:N relationship) with attributes: Rating, ReviewText, ReviewDate.

Part 4: Normalization Workshop

Task 4.1: Denormalized Table Analysis

Given Table: StudentProject (StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

1. Functional Dependencies (FDs):

- FD1: StudentID → StudentName, StudentMajor
- FD2: ProjectID → ProjectTitle, ProjectType, SupervisorID
- FD3: SupervisorID → SupervisorName, SupervisorDept
- FD4: {StudentID, ProjectID} → Role, HoursWorked, StartDate, EndDate

2. Anomalies & Redundancy:

- **Redundancy:** Student details and Supervisor details are repeated across multiple project entries.
- **Update Anomaly:** Changing a supervisor's department requires updating every row where that supervisor appears.
- **Insert Anomaly:** Cannot record a new project without assigning a student to it first.
- **Delete Anomaly:** Removing the only student from a project accidentally deletes the project's details entirely.

3. 1NF Analysis:

The table is in 1NF because all attributes contain atomic values.

4. 2NF Decomposition (Removing Partial Dependencies):

Primary Key of original table: {StudentID, ProjectID}

- **Student:** (StudentID, StudentName, StudentMajor)
- **Project:** (ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept)
- **StudentProjectAssignment:** (StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

5. 3NF Decomposition (Removing Transitive Dependencies):

FD3 (SupervisorID → SupervisorName, SupervisorDept) represents a transitive dependency in the Project table.

Final 3NF Schemas:

- **Student:** (StudentID, StudentName, StudentMajor)
- **Supervisor:** (SupervisorID, SupervisorName, SupervisorDept)
- **Project:** (ProjectID, ProjectTitle, ProjectType, *SupervisorID*)
- **StudentProjectAssignment:** (StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

Task 4.2: Advanced Normalization

Given Table: CourseSchedule (StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

1. **Primary Key:** {StudentID, TimeSlot, Room} (or {StudentID, CourseID})
 2. **Functional Dependencies:**
 - StudentID → StudentMajor
 - CourseID → CourseName
 - InstructorID → InstructorName
 - Room → Building
 - {CourseID, TimeSlot} → InstructorID, Room
 3. **BCNF Decomposition:**
 - **Student:** (StudentID, StudentMajor)
 - **Course:** (CourseID, CourseName)
 - **Instructor:** (InstructorID, InstructorName)
 - **RoomLocation:** (Room, Building)
 - **Section:** (CourseID, TimeSlot, InstructorID, Room)
 - **Enrollment:** (StudentID, CourseID)
-

Part 5: Design Challenge

Task 5.1: Real-World Application (Student Clubs)

1. Relational Schema:

- **Student:** (StudentID, Name, Major, Email)
- **Club:** (ClubID, ClubName, Category, *FacultyAdvisorID*)
- **FacultyAdvisor:** (AdvisorID, Name, Department)
- **Membership:** (StudentID, ClubID, JoinDate, Role)
- **Event:** (EventID, EventName, EventDate, *ClubID*, *RoomID*)
- **RoomReservation:** (RoomID, RoomNumber, Building, Capacity)

2. Example Queries (English Format):

1. "Find all students who hold officer positions (e.g., President, Treasurer) in the Computer Science Club."
2. "List all upcoming events scheduled for next week along with their assigned room number and building location."
3. "Retrieve the total budget expenses incurred by each student club for the current academic semester."